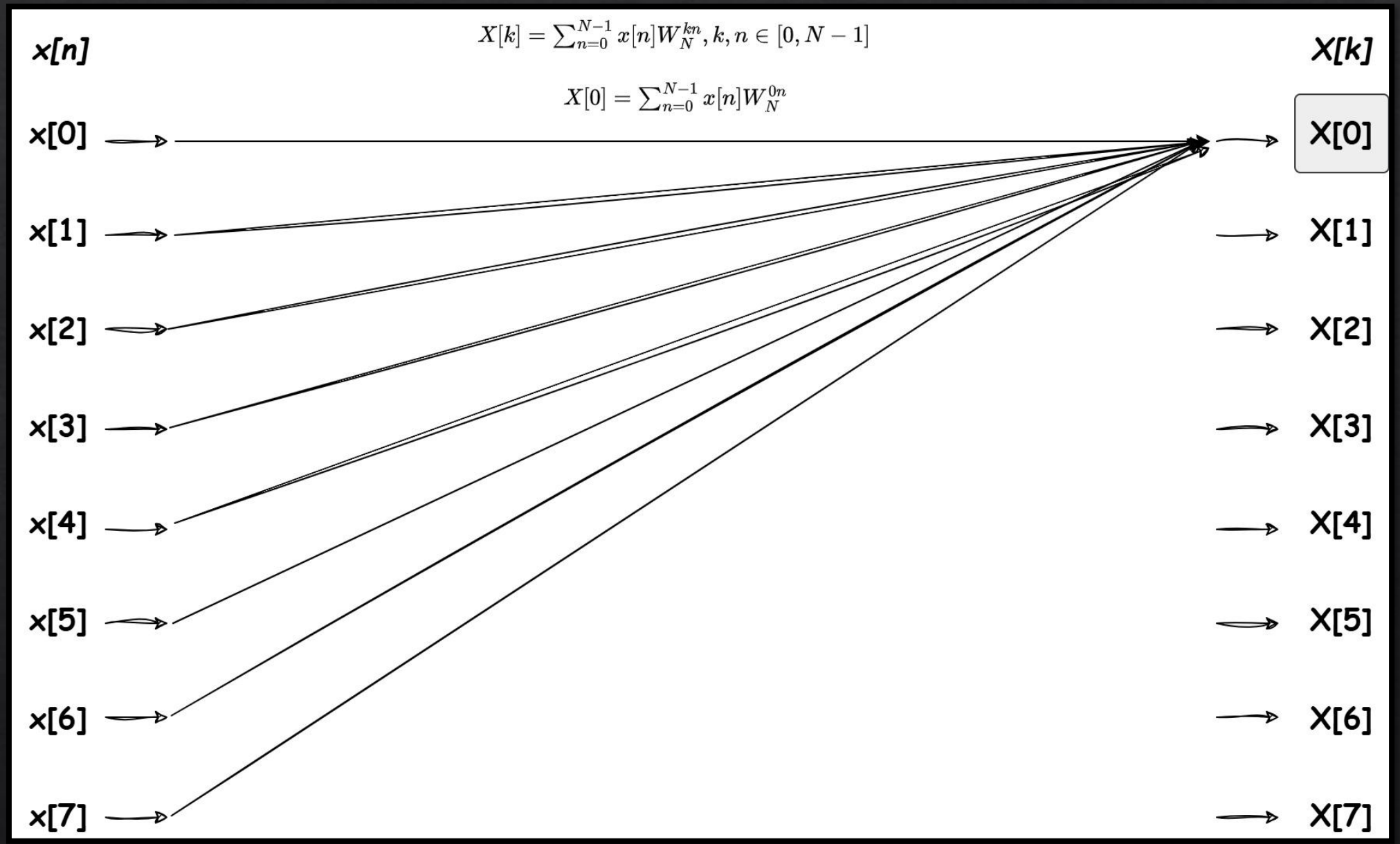
Assumptions about audience of this presentation:

- At least a basic understanding of what frequency domain analysis means.
- Aware of the most basic FFT algebraic formulation: $X[k] = \sum_{j=0}^{N-1} x[n]\, e^{-2\pi ik/N}\ for\ k \in [0, N-1]$ and that $W_N = e^{-2\pi ik/N}$
- Basic knowledge of non-shared memory parallel computing paradigms like MPI
- Basic understanding of shared memory parallel computing paradigms like pthreads, to understand the CUDA implementation

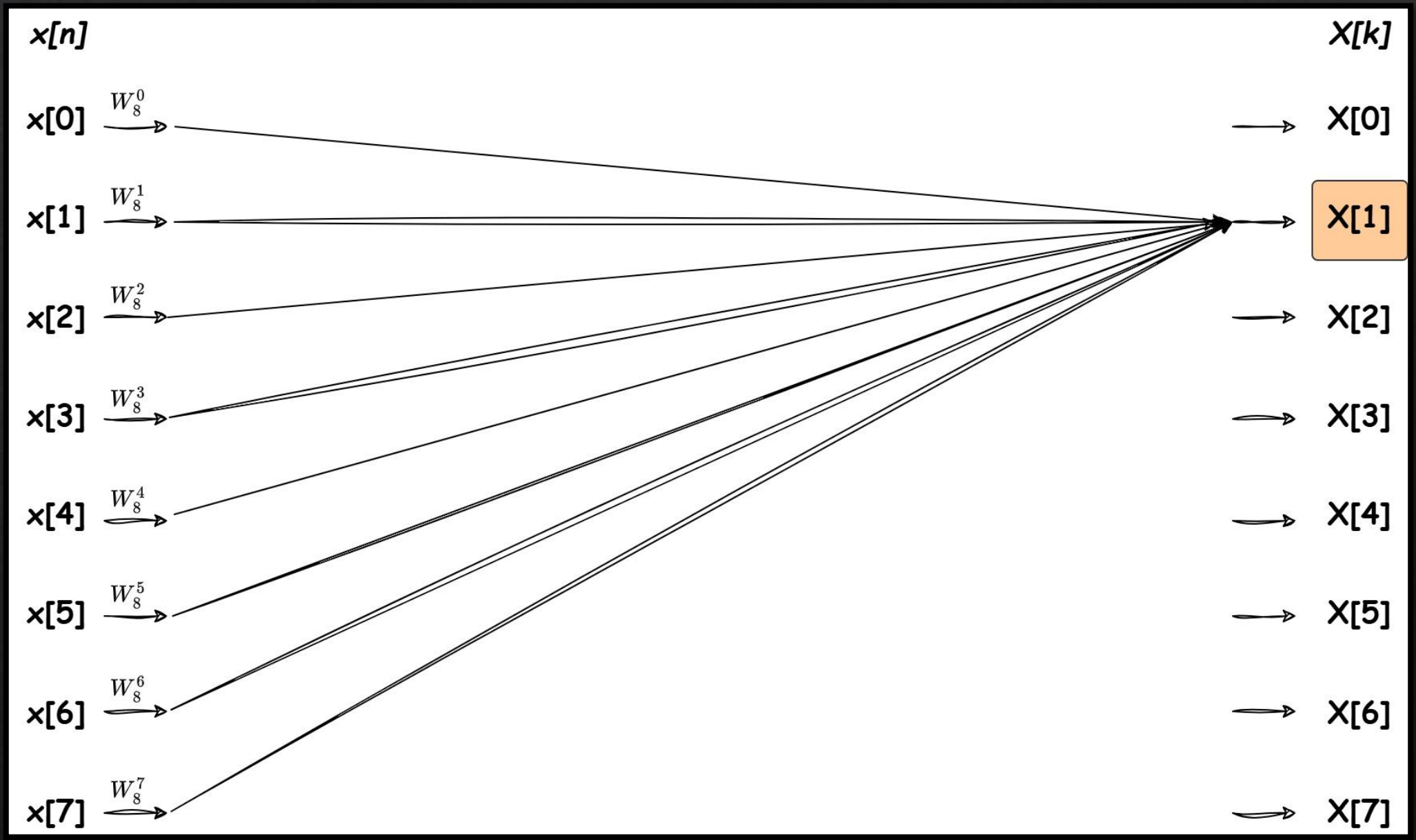
DFT – Computation Visualization
x[n] – time domain X[k] – frequency domain

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, k, n \in [0, N-1]$$

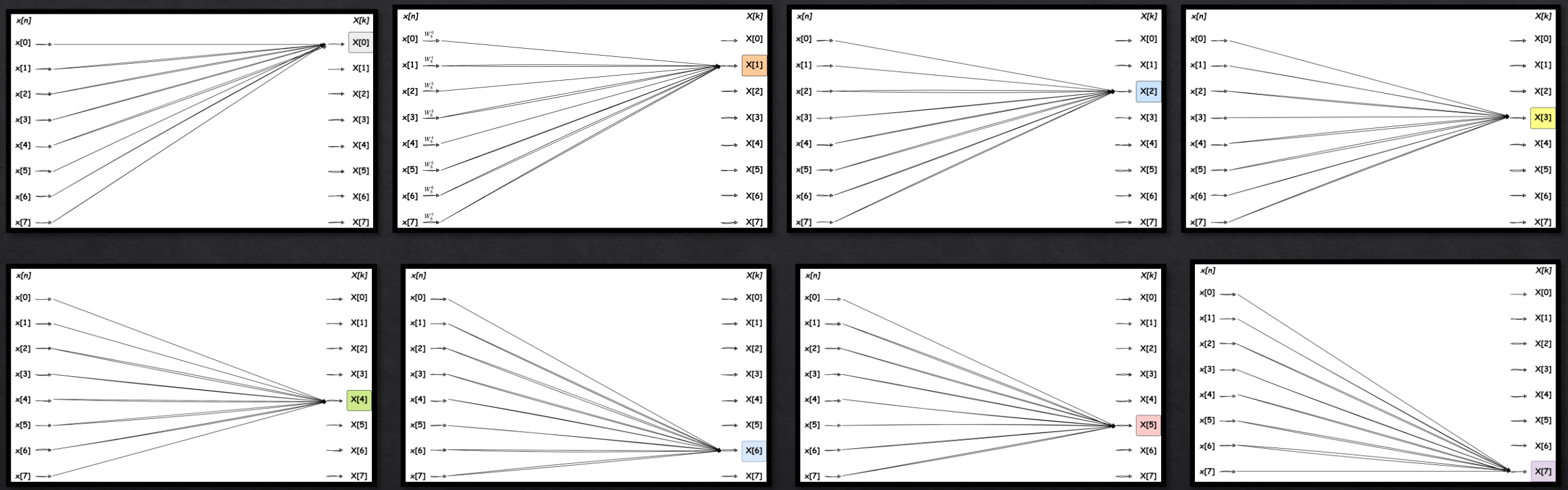
$$X[0] = \sum_{n=0}^{N-1} x[n] W_N^{0n}$$



DFT – Computation Visualization
x[n] – time domain X[k] – frequency domain



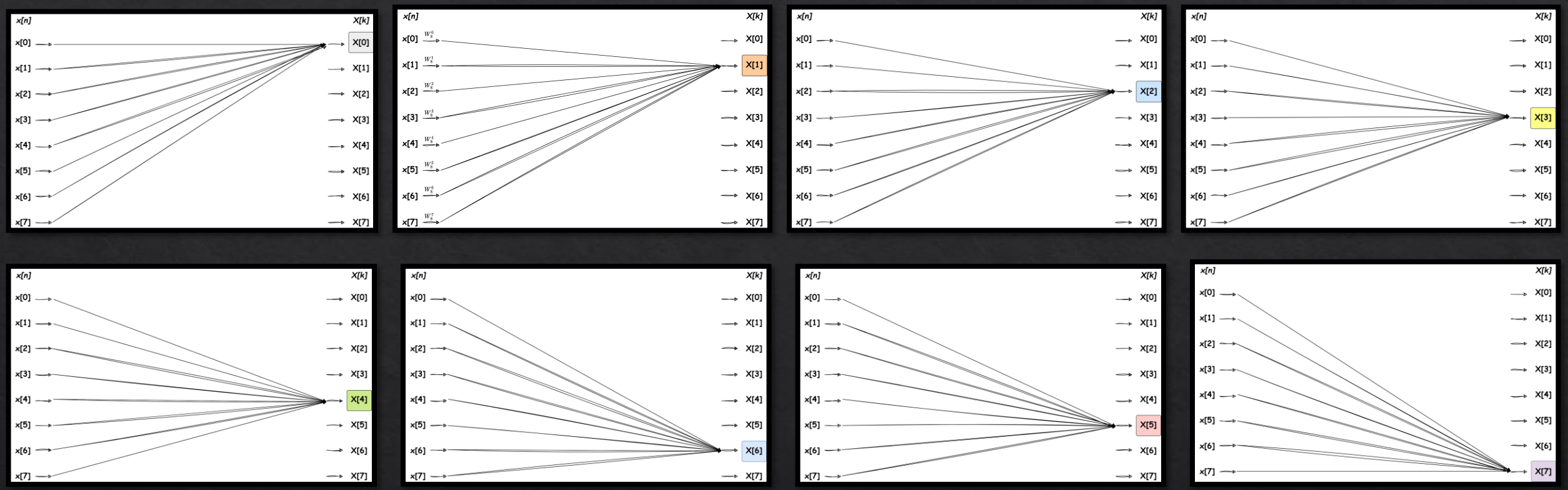
DFT – Computation Visualization – O(N^2)
x[n] – time domain X[k] – frequency domain



Computation cost for every X[k] is N complex multiplication and N-1 complex additions.

Computational cost for each complex addition – 2 real adds/subs: $(a + bi) \pm (c + di) = (a \pm c) + (b \pm d)i$ [2 flops]
Computation cost for each complex multiplication – 4 real mults + 2 real adds/subs: $(a + bi)(c + di) = (ac - bd) + (ad + ab)i$ [6 flops]

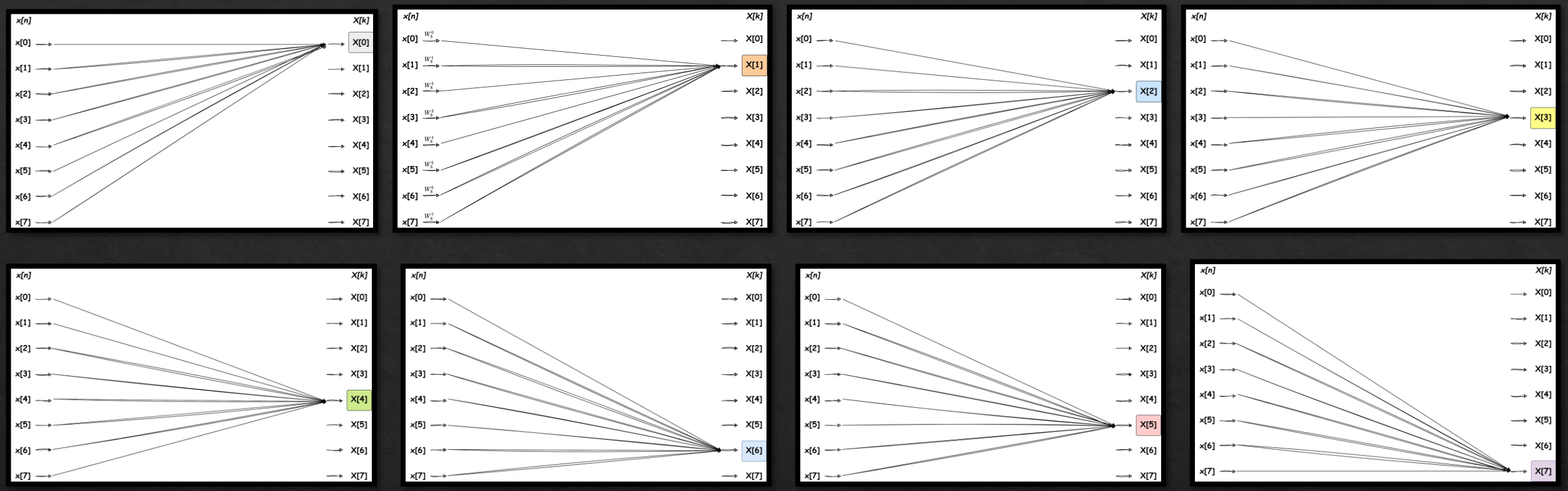
DFT – Computation Visualization – O(N^2)
x[n] – time domain X[k] – frequency domain



Computation cost for every X[k] is N complex multiplication and N-1 complex additions.

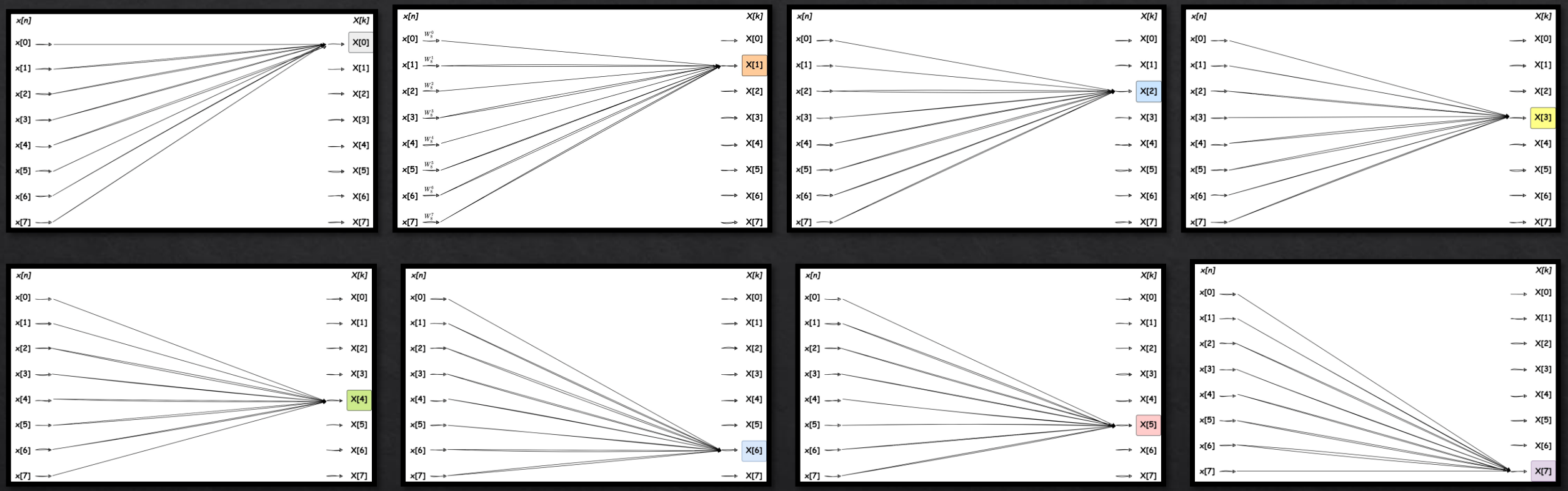
- Complex addition/subtraction: 2 flops.
- Complex multiplication: 6 flops
- Every element of X[k] takes 6N+2(N-1)=8N-2 flops
- In order to compute all of X[k], it takes N(N*(8N-2)) flops -> O(N^2)

DFT – Computation Visualization – O(N^2)
x[n] – time domain X[k] – frequency domain



Computation cost of DFT is O(N^2)

Start thinking about FFT – How can we start to optimize?



Find overlapping computations. Use symmetry properties of the Fourier basis $e^{-2\pi k/N}, k \in [0, N-1]$

The algorithm is called Fast Fourier Transform – more specifically Decimation in Time (as opposed to another version, Decimation in Frequency) – more specifically radix-2 FFT.

- Invented by Gauss 1805 (not recognized)
- Danielson and Lanczos 1942
- Cooley & Tookey – IBM 1965 – For the digital eta. Open and source and mixed-radix implementations.

2 central algorithmic improvements:

- Remove redundant calculations in between 'stages'
- Remove redundant calculations inside every stage (butterfly operations).

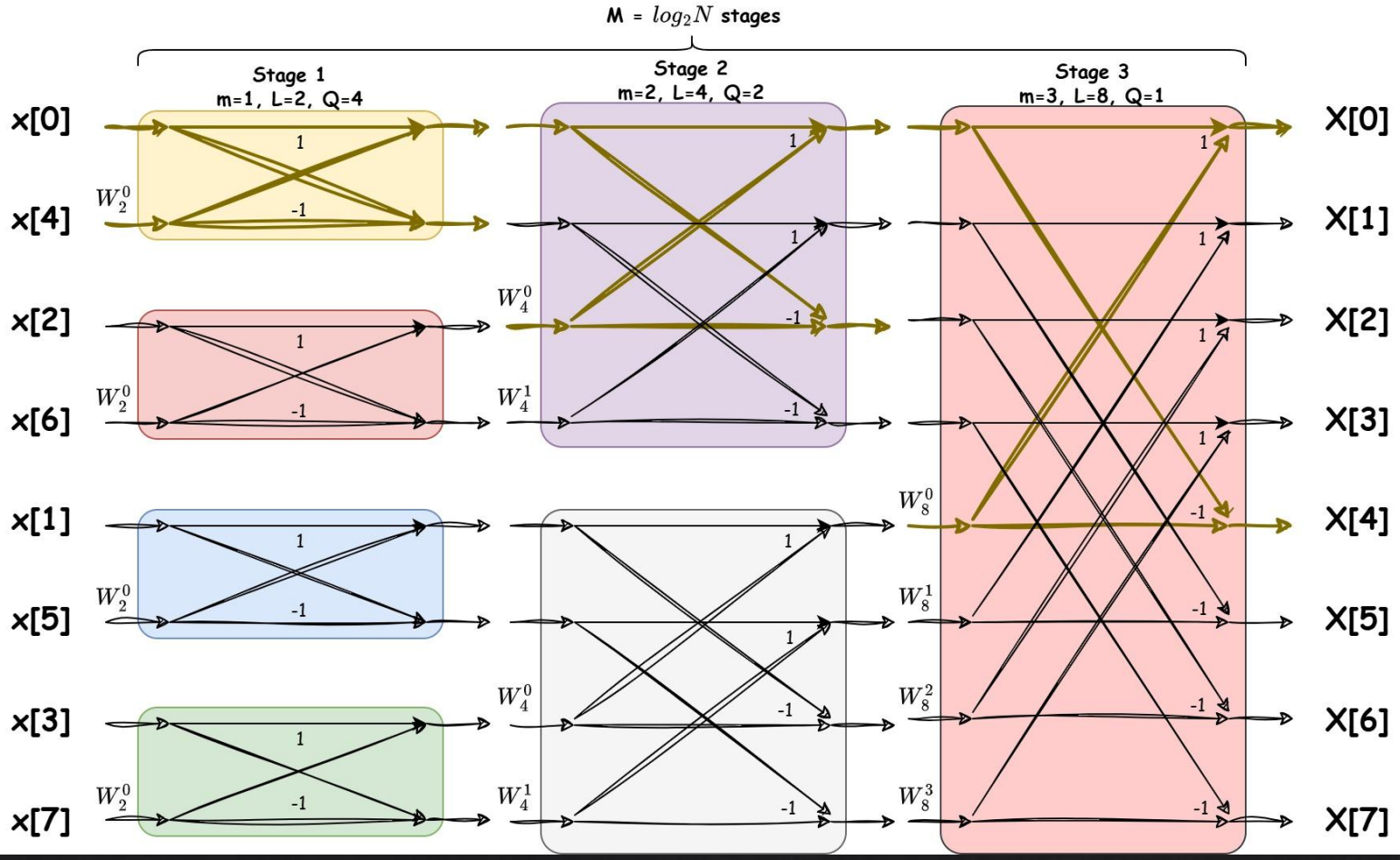
$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, k, n \in [0, N-1]$$

$$X[k] = \sum_{r=0}^{\frac{N}{2}-1} x[2r] W_{\frac{N}{2}}^{kr} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x[2r+1] W_{\frac{N}{2}}^{kr}$$

$$X[k] = \underbrace{G[k]}_{\text{even}} + W_N^k \underbrace{H[k]}_{\text{odd}}$$

** Decimate until signals are of size 2

$$X[k] = \underbrace{[G_1[k]}_{\text{even}} + W_{\frac{N}{2}}^k \underbrace{G_2[k]]}_{\text{odd}} + W_N^k \underbrace{[H_1[k]}_{\text{even}} + W_{\frac{N}{2}}^k \underbrace{H_2[k]]}_{\text{odd}}$$



2 central algorithmic improvements from DFT:

- Remove redundant calculations in between ‘stages’
- Remove redundant calculations inside every stage (butterfly operations).

Notice:

- Bit reversed order of x[n] signal at input
- Butterfly Operations
- Notice signal is of length 2^something (this is radix-2)

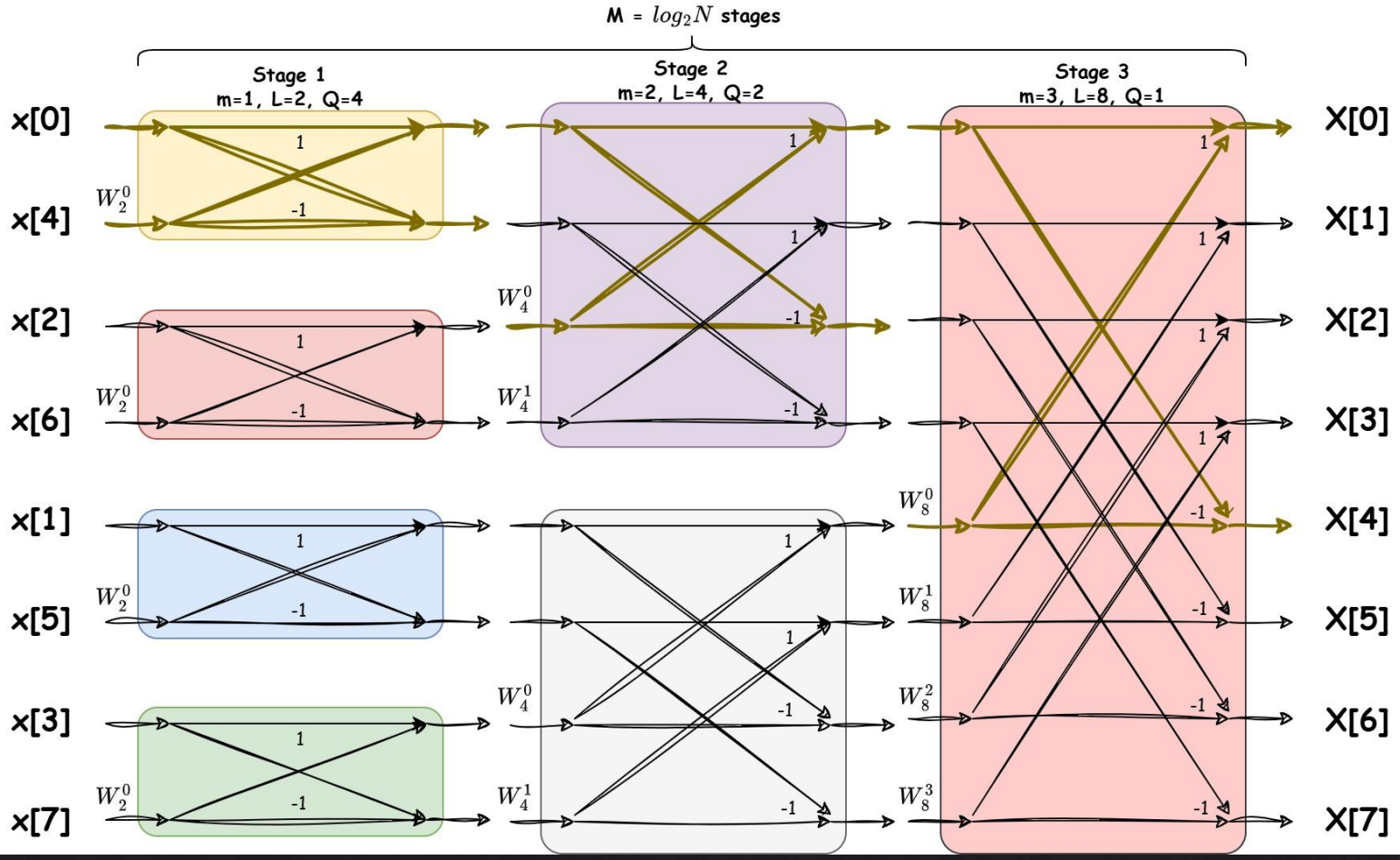
$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, k, n \in [0, N-1]$$

$$X[k] = \sum_{r=0}^{\frac{N}{2}-1} x[2r] W_{\frac{N}{2}}^{kr} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x[2r+1] W_{\frac{N}{2}}^{kr}$$

$$X[k] = \underbrace{G[k]}_{\text{even}} + W_N^k \underbrace{H[k]}_{\text{odd}}$$

** Decimate until signals are of size 2

$$X[k] = \underbrace{[G_1[k]}_{\text{even}} + W_{\frac{N}{2}}^k \underbrace{G_2[k]]}_{\text{odd}} + W_N^k \underbrace{[H_1[k]}_{\text{even}} + W_{\frac{N}{2}}^k \underbrace{H_2[k]]}_{\text{odd}}$$



Runtime Improvement!

Why is the runtime O(NlogN) ?

- Every butterfly has 1 complex mult and 2 additions
- There are N / 2 butterflies per stage
- The are logN stages
- Overall –> O(NlogN)

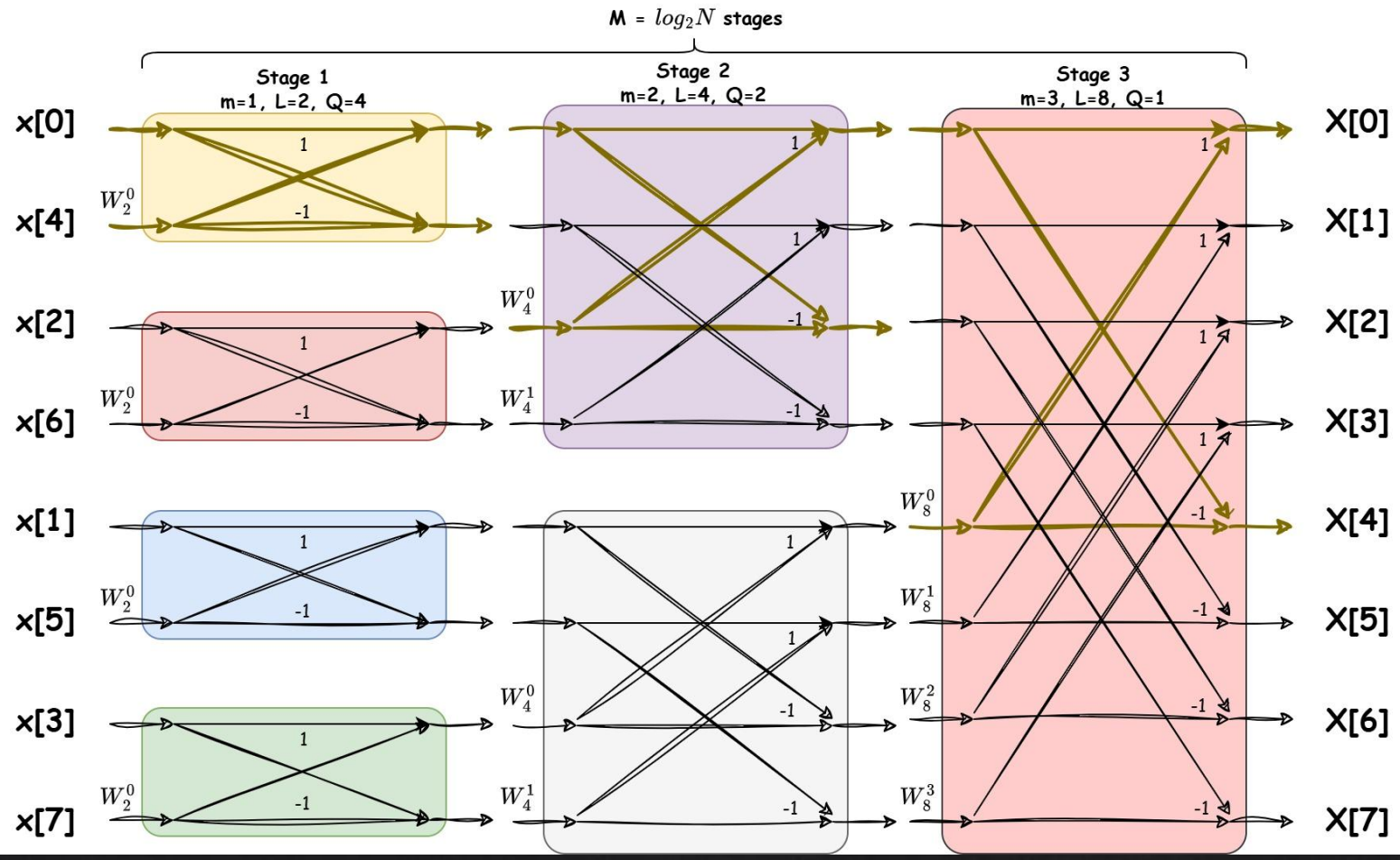
$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, k, n \in [0, N-1]$$

$$X[k] = \sum_{r=0}^{\frac{N}{2}-1} x[2r] W_{\frac{N}{2}}^{kr} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x[2r+1] W_{\frac{N}{2}}^{kr}$$

$$X[k] = \underbrace{G[k]}_{\text{even}} + W_N^k \underbrace{H[k]}_{\text{odd}}$$

** Decimate until signals are of size 2

$$X[k] = \underbrace{[G_1[k]}_{\text{even}} + W_{\frac{N}{2}}^k \underbrace{G_2[k]]}_{\text{odd}} + W_N^k \underbrace{[H_1[k]}_{\text{even}} + W_{\frac{N}{2}}^k \underbrace{H_2[k]]}_{\text{odd}}$$



High-Performance and Parallel FFT considerations

Considerations:

Data Level Parallelism

- Complex data (real+complex) layout in computer memory (SoA vs AoS vs AoSoA)
- SIMD Complex multiplication operations

Algorithm Level Parallelism:

- High-Level Algorithm – computational complexity, big O
- 2 sub algorithms: (a) Data bit-reversal ordering, and (b) Decimation in Time butterfly stages.
- Data access pattern informed data distribution. Blocked vs cyclical vs group-cyclical

Data Level Parallelism: Complex Data (real + imag)

Option 1 – AoS (Array of Structures) WORST

Non-contiguous real/imaginary signal. Bad for loading into SIMD quickly.

```
struct complex_num {
    float real;
    float imaginary;
};

struct complex_signal[N];
```

Option 2 – SoA (Structure of Arrays) BETTER

Pros:
Contiguous real/imaginary signal. Better for loading data into SIMD hardware.

Cons: real and imaginary components of each single number have a distance of size N from each other. For large N very bad for locality.

```
typedef struct {
    float* real[N];
    float imag[N];
} ComplexSoA;
```

Data Level Parallelism: Complex Data (real + imag)

Option 3 – AoSoA (Array of Structure of Arrays) BEST

Pro:
Contiguous data blocks small enough to exploit cache locality.

Cons:
More complicated to work with.

```
// Define the structure for a single block of complex numbers
typedef struct {
    float real[BLOCK_SIZE];
    float imag[BLOCK_SIZE];
} ComplexBlock;

// Define the structure for the AoSoA
typedef struct {
    ComplexBlock *blocks;  // Dynamically allocated array of blocks
    int num_blocks;        // Total number of blocks
} ComplexAoSoA;
```

Option 3 – AoSoA (Array of Structure of Arrays) BEST

Pro:
Contiguous data blocks small enough to exploit cache locality.

Cons:
More complicated to work with.

More Data Level Parallelism: Use SIMD (AVX2)

With Intel AVX2, I can use the YMM vector registers (__m256) which have 256 bit-width fields.

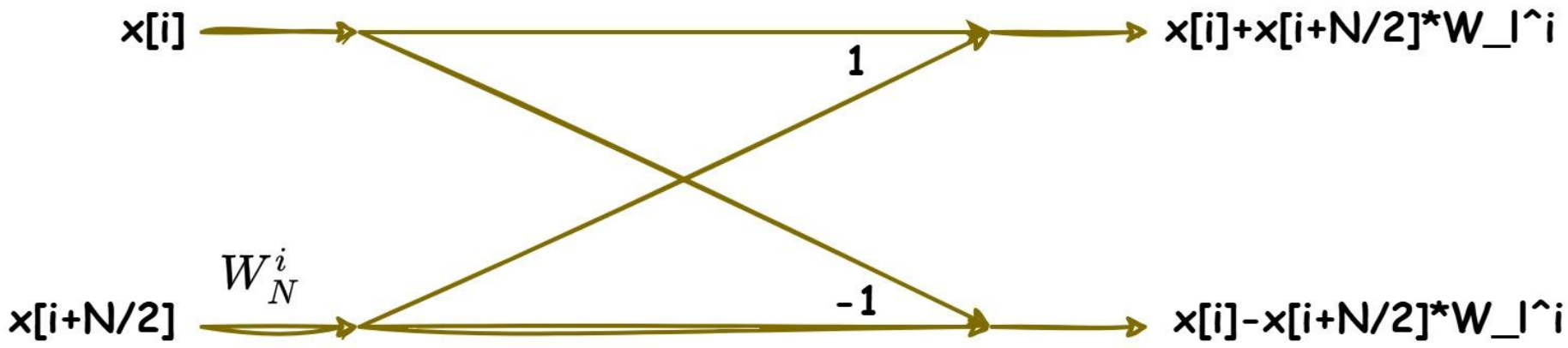
For complex data where each of the real and complex components are floats (4 bytes each = 32 bits), then each YMM register can hold 256/32=8 complex numbers, or 16 real (or complex) values.

Butterfly Operation - Why +/-1?

The 'distance' between every 2 elements in a butterfly operation is $\frac{N}{2}$.

Because of that we take advantage of the fact that $W_N^n = -W_N^{n+\frac{N}{2}}$.

This reduces a butterfly operation from 2 complex multiplications and 2 complex additions/subtractions (16 flops) to 1 complex multiplcation and 2 adds/subs (10 flops).



Butterfly Operations - explanation

Radix-2 FFT Computational Cost – O(NlogN)

Radix-4 FFT Computational Cost – O(NlogN)

Radix-4 FFT

```
======N:4, P:2======

Stage 1 Distribution BLOCK! with block size 2
Processor 0: [0, 1]
Processor 1: [2, 3]

Stage 2 Distribution CYCLIC! with cycle 2
Processor 0: [0, 2]
Processor 1: [1, 3]
```

```
=== Parallel FFT Distribution Visualization ===
n=4, p=2.  We track k in powers of 2, do partial redistrib if c < p.

k=2  c=1:
| 0(p0) | 1(p0) | 2(p1) | 3(p1) |
    |       |       |       |
    |       |       |       |
    v       v       v       v

k=4  c=2:
| 0(p0) | 1(p1) | 2(p0) | 3(p1) |
Number of redistributions: 1
Done.
```